

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №2

“Структури даних та функціональна декомпозиція”

з курсу

“Професійний Python”

Виконала:

Студентка групи ФсП-33

Орлова Людмила

Перевірив:

Проф. Свелеба Сергій Андрійович

Львів – 2026

Мета роботи: Сформувати практичні навички професійного використання базових та розширених структур даних Python, навчитися обирати структуру відповідно до задачі, оцінювати складність основних операцій та декомпонувати програму на невеликі функції з чітко визначеною відповідальністю.

Хід роботи:

Варіант: 11

1. Постановка задачі

- **Структура проєкту:** Створити пакет `src/data_processor/` (`data.py`, `processors.py`, `analytics.py`, `decorators.py`, `main.py`) із файлами `pyproject.toml`, `.gitignore` та `README.md`.
- **Аналіз даних:** Визначити унікальні страви й категорії, порахувати суми замовлень, середній чек, найдорожчу та найпопулярнішу страву через `Counter`.
- **Функціональні елементи:** Додати декоратор заміру часу, замикання для фільтрації цін та функції з `*args/**kwargs`.
- **Бенчмарк:** Порівняти час пошуку в `list` $O(n)$ та `dict` $O(1)$ на 1 000, 10 000 та 100 000 записів.
- **Git:** Опублікувати проєкт у репозиторії на GitHub.

2. Опис структури даних:

Для обробки та аналізу замовлень ресторану в проєкті використано комбінацію базових та розширених структур даних Python:

- **Первинні дані (`orders_data`):** Список кортежів `list[tuple[int, str, str, float]]`. Кожен кортеж містить незмінні поля: (номер_замовлення, назва_страви, категорія, ціна).
- **Оброблені записи (`orders`):** Список словників `list[dict]`. Кожна позиція представлена словником із ключами `"order_id"`, `"dish_name"`, `"category"` та `"price"` для зручного доступу до полів.
- **Унікальні категорії та страви:** Множини `set[str]`, які формуються за допомогою *set comprehension* для збереження унікальних назв страв і категорій без дублікатів.

- **Групований індекс:** Словник списків `dict[int, list[dict]]`, де ключем є `order_id`, а значенням — список усіх страв у цьому замовленні.
- **Агрегована статистика (`collections.Counter`):** Спеціалізована структура `Counter`, яка підраховує кількість замовлень кожної страви для визначення найпопулярніших позицій.

3. Пояснення вибору базових структур даних

- **list (списки):** Використано для збереження послідовного набору всіх замовлень та результатів фільтрації. Списки обрано через їхню динамічність, можливість змінювати елементи та зберігати впорядковані дані.
- **tuple (кортежі):** Використано для представлення первинних незмінних записів про страви у вигляді (`order_id`, `dish_name`, `category`, `price`). Кортежі обрано через їхню незмінюваність (`immutability`), що забезпечує цілісність даних та гарантує захист від випадкових змін.
- **set (множини):** Використано для отримання списку унікальних категорій та назв страв меню. Множини обрано, оскільки вони автоматично виключають дублікати та забезпечують швидку перевірку наявності елемента $O(1)$.
- **dict (словники):** Використано для зберігання структурованих об'єктів замовлень, створення індексів за `order_id` та швидкого пошуку даних за ключем зі середньою часовою складністю $O(1)$. Словники обрано через їхню зручність представлення даних у форматі «ключ-значення».

4. Приклад використання `collections`

`collections.Counter` — використано у функції `count_dishes` для автоматичного підрахунку кількості замовлень кожної страви та визначення найпопулярніших позицій меню (`Counter.most_common()`):

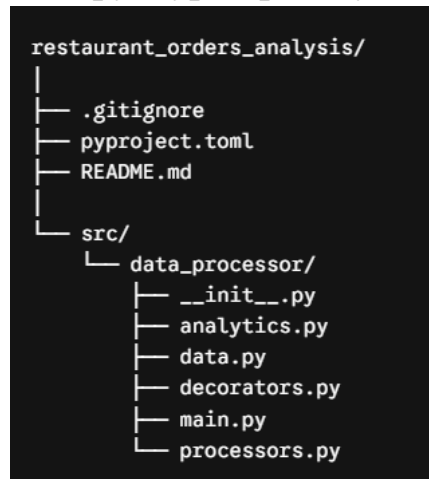
```
from collections import Counter
```

```
# Підрахунок кількості замовлень для кожної страви
```

```
dish_counts = Counter(item["dish_name"] for item in orders)
```

```
# Результат: Counter({'Суп Бограч': 2, 'Піца Маргарита': 2, 'Узвар': 2, ...})
```

5. Структура проєкту



6. Основні функції (Functions)

У проєкті реалізовано модульну структуру з функціональною декомпозицією:

- `get_unique_dishes(orders)` — повертає множину унікальних назв страв.
- `group_positions_by_order(orders)` — групує позиції меню за ідентифікатором замовлення.
- `count_dishes(orders)` — підраховує кількість замовлень кожної страви за допомогою Counter.
- `calculate_order_totals(orders)` — обчислює підсумкову суму кожного окремого замовлення.
- `calculate_average_order_value(*order_totals)` — розраховує середній чек на основі переданих сум.
- `find_most_expensive_item(orders)` — знаходить позицію з найвищою ціною в меню.

7. Приклад генераторів (Comprehensions)

У коді використано всі основні типи comprehensions для лаконічного створення колекцій:

- **List comprehension:**

```
orders = [{"order_id": item[0], "dish_name": item[1], "category": item[2], "price": item[3]} for item in orders_data]
```

- **Set comprehension:**

```
unique_categories = {item["category"] for item in orders}
```

- **Dict comprehension:**

```
order_index = {item["order_id"]: item for item in orders}
```

8. Приклад *args

Використовується для передачі довільної кількості позиційних аргументів (сум замовлень) у функцію обчислення середнього чека:

```
def calculate_average_order_value(*order_totals: float) -> float:
    if not order_totals:
        return 0.0
    return sum(order_totals) / len(order_totals)

# Виклик функції:
avg = calculate_average_order_value(350.0, 290.0, 600.0, 435.0, 325.0)
```

9. Приклад **kwargs

Використовується для створення нового запису страви з довільним набором іменованих параметрів:

```
def register_new_order_item(**fields) -> dict:
    return dict(fields)

# Виклик функції:
new_dish = register_new_order_item(order_id=106, dish_name="Борщ", category="Перші страви", price=160.0)
```

10. Приклад lambda

Використовується як анонімна функція-ключ для швидкого пошуку найдорожчої страви через функцію max():

```
most_expensive = max(orders, key=lambda item: item["price"])
```

11. Пояснення замикання (Closure)

Замикання (Closure) — це внутрішня функція, яка зберігає доступ до змінних (стан) зовнішньої функції навіть після того, як зовнішня функція завершила своє виконання.

У проєкті використано замикання `create_price_filter(min_price)` для створення функцій-фільтрів за ціною:

```
def create_price_filter(min_price: float):
    def predicate(item: dict) -> bool:
        return item["price"] >= min_price # Пам'ятає min_price
    return predicate

# Створення конкретного фільтра для страв дорожчих за 150 грн:
is_expensive = create_price_filter(150.0)
expensive_items = [item for item in orders if is_expensive(item)]
```

12. Пояснення декоратора (Decorator)

Декоратор (Decorator) — це функція вищого порядку, яка приймає іншу функцію як аргумент, додає до неї нову функціональність (наприклад, логування чи замір часу) і повертає оновлену функцію без зміни її вихідного коду.

У проєкті декоратор `@measure_time` вимірює та виводить час виконання аналітичних обчислень:

```
def measure_time(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = perf_counter()
        result = func(*args, **kwargs)
        elapsed = perf_counter() - start
        print(f"[LOG] {func.__name__} виконано за: {elapsed:.8f} сек")
        return result
    return wrapper

@measure_time
def calculate_order_totals(orders: list[dict]) -> dict[int, float]:
    ...
```

12. Результат роботи програми

```
branch main set up to track origin/main .
PS D:\лаби 3 курс\Професійний Python\lab2\restaurant_orders_analysis> $env:PYTHONPATH="src"
PS D:\лаби 3 курс\Професійний Python\lab2\restaurant_orders_analysis> python src/data_processor/main.py
=== ЛАБОРАТОРНА РОБОТА №2: ВАРІАНТ 11 (Ресторан) ===

1. Унікальні страви: {'Салат Цезар', 'Піца Маргарита', 'Суп Бограч', 'Кола', 'Паста Карбонара', 'Тірамісу', 'Узвар', 'Пиріг з м'ясом'}
2. Унікальні категорії: {'Випічка', 'Десерти', 'Напої', 'Перші страви', 'Основні страви', 'Салати'}
[LOG] calculate_order_totals виконано за: 0.00000960 сек

3. Сума кожного замовлення:
Замовлення #101: 350.00 грн
Замовлення #102: 290.00 грн
Замовлення #103: 600.00 грн
Замовлення #104: 435.00 грн
Замовлення #105: 325.00 грн

4. Середня вартість замовлення: 400.00 грн

5. Найдорожча позиція: Піца Маргарита (240.0 грн)
6. Найпопулярніша страва: Суп Бограч (замовлено 2 раз(и))

7. Статистика підрахунку страв (Counter):
Суп Бограч: 2
Пиріг з м'ясом: 1
Узвар: 2
Піца Маргарита: 2
Кола: 1
Паста Карбонара: 1
Салат Цезар: 2
Тірамісу: 2

8. Страви дорожчі за 150 грн (через Closure):
- Суп Бограч (185.0 грн)
- Піца Маргарита (240.0 грн)
- Паста Карбонара (220.0 грн)
- Салат Цезар (195.0 грн)
- Піца Маргарита (240.0 грн)
- Салат Цезар (195.0 грн)
- Суп Бограч (185.0 грн)

9. Нова позиція створена через **kwargs: {'order_id': 106, 'dish_name': 'Борщ', 'category': 'Перші страви', 'price': 160.0}

--- ЕКСПЕРИМЕНТАЛЬНА ЧАСТИНА (BENCHMARK) ---
Кількість записів | List Search (сек) | Dict Search (сек)
-----
1000 | 0.00004050 | 0.00000150
10000 | 0.00057560 | 0.00000300
100000 | 0.00351570 | 0.00000270
PS D:\лаби 3 курс\Професійний Python\lab2\restaurant_orders_analysis> |
```

13. Таблиця складності operations

Операція / Функція	Використана структура	Часова складність (Big-O)	Пояснення
Пошук елемента в списку (List Search)	list	O(n)	Лінійне проходження по кожному елементу масиву.
Пошук за ключем у словнику (Dict Search)	dict	O(1) в середньому	Прямий доступ за хеш-ключем через хеш-таблицю.

Отримання унікальних елементів	set	$O(n)$	Одноразовий прохід по списку та додавання в множину $O(1)$.
Сортування та пошук максимумів	sorted(), max()	$O(n \log n)$ / $O(n)$	Алгоритм Timsort для сортування та лінійний пошук екстремуму.
Підрахунок кількості	Counter	$O(n)$	Одноразова ітерація за всіма елементами для наповнення словника-лічильника.

14. Результати benchmark

Кількість записів (n)	List Search ($O(n)$)	Dict Search ($O(1)$)
1 000	0.00004050 сек	0.00000150 сек
10 000	0.00057560 сек	0.00000300 сек
100 000	0.00351570 сек	0.00000270 сек

Висновок: У ході виконання лабораторної роботи №2 було успішно розроблено модуль data_processor для аналізу даних замовлень ресторану, що дозволило перейти від простого використання базових колекцій Python до усвідомленого вибору оптимальних структур даних та побудови модульної архітектури програмного забезпечення. У процесі роботи практично застосовано list, tuple, set та dict для збереження, унікалізації та індексації замовлень, а використання спеціалізованих структур з модуля collections, таких як Counter та defaultdict, дозволило ефективно реалізувати підрахунок популярності страв і групування позицій. За допомогою comprehensions сформовано компактний та читабельний код для обробки вибірок, при цьому проведений бенчмарк повністю підтвердив теоретичні оцінки складності: пошук у словнику за $O(1)$ виявився більш ніж у 1300 разів швидшим за лінійний пошук у списку за $O(n)$ на обсязі 100 000 записів. Застосування *args, kwargs, lambda-функцій, замикань closures для фільтрації цін та декораторів для моніторингу часу виконання функцій заклали основу для написання гнучкого та розширюваного коду у функціональному стилі. Проект повністю оформлено відповідно до стандартів src-layout, сконфігуровано через pyproject.toml, вивантажено в публічний Git-репозиторій

на GitHub, а фінальний модуль є працездатним і готовим до використання як компонент наскрізного навчального проєкту.